

Lab 2 - Wstęp do klasyfikacji

1. Wstęp

Podczas pierwszych laboratoriów przedstawione zostały podstawowe zagadnienia związane z klasyfikacją. Omówiono proces przygotowania danych, w tym:

- Czyszczenie i uzupełnianie brakujących wartości,
- Skalowanie i normalizację cech,
- Radzenie sobie z niezbalansowanymi zbiorami,
- Wybór cech i ich inżynierię.

Na zakończenie zajęć został zbudowany pierwszy, bazowy model klasyfikacyjny, który posłuży jako punkt odniesienia do dalszej optymalizacji.

Laboratoria numer 2 poświęcone będą optymalizacji modeli oraz ocenie ich skuteczności. Poruszone zostaną następujące zagadnienia: Metody oceny skuteczności, Dobór modeli klasyfikacyjnych, Dostrajanie hiperparametrów, Wizualizacja wyników, Analiza istotności cech.

Po zakończeniu zajęć uczestnicy będą potrafili skutecznie dostrajać i oceniać modele klasyfikacyjne, wybierać odpowiednie algorytmy oraz interpretować uzyskane wyniki.

2. Metody oceny skuteczności

Do oceny skuteczności wykorzystuje się do tego różne metryki, które mierzą jakość predykcji w zależności od zastosowania i charakterystyki danych.

Macierz pomyłek to tablica przedstawiająca liczbę prawidłowych i błędnych klasyfikacji dla każdej klasy w problemie klasyfikacyjnym. Dla klasyfikacji binarnej ma postać:

		Klasa rzeczywista		
		Pozytywna	Negatywna	
Klasa przewidywana	Pozytywna	Prawdziwie pozytywna (TP)	Fałszywie pozytywna (FP)	<u>Recall</u> $TP / (TP + FN)$
	Negatywna	Fałszywie negatywna (FN)	Prawdziwie negatywna (TN)	
<u>Accuracy</u> $(TP + TN) / (TP + TN + FP + FN)$		<u>Precision</u> $TP / (TP + FP)$		

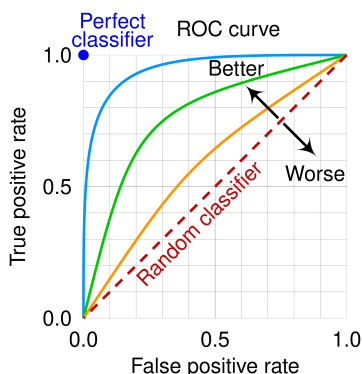
Interpretacja wartości w macierzy:

- True Positive (TP) – liczba poprawnie sklasyfikowanych przykładów pozytywnych.
- False Positive (FP, błąd I rodzaju) – liczba przykładów negatywnych błędnie sklasyfikowanych jako pozytywne.
- False Negative (FN, błąd II rodzaju) – liczba przykładów pozytywnych błędnie sklasyfikowanych jako negatywne.
- True Negative (TN) – liczba poprawnie sklasyfikowanych przykładów negatywnych.

Metryki klasyfikacyjne:

Accuracy (Dokładność) $Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$	Dokładność to najprostsza miara skuteczności klasyfikatora, określająca, jaki procent próbek został poprawnie sklasyfikowany.	Intuicyjna i łatwa do interpretacji. Sprawdza się w dobrze zbalansowanych zbiorach danych. Może być myląca dla niezbalansowanych danych
Precision (Precyzja) $Precision = \frac{TP}{TP + FP}$	Precyzja określa, jaki procent próbek przewidzianych jako pozytywne rzeczywiście należy do klasy pozytywnej.	Wysoka precyzja oznacza, że model rzadko przypisuje błędnie negatywne przykłady do klasy pozytywnej (mało FP). Ważna w aplikacjach, gdzie koszt błędu FP jest wysoki (np. diagnostyka chorób, wykrywanie oszustw).
Recall (Czułość) $Recall = \frac{TP}{TP + FN}$	Recall określa, jaki procent rzeczywistych przypadków pozytywnych został poprawnie wykryty.	Wysoka wartość recall oznacza, że model wykrywa prawie wszystkie pozytywne przypadki (mało FN). Ważna w aplikacjach, gdzie przeoczenie pozytywnego przypadku jest kosztowne (np. wykrywanie nowotworów).
F1-score $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$	F1-score to średnia harmoniczna precision i recall.	F1-score równoważy precision i recall, dzięki czemu jest użyteczny w niezbalansowanych zbiorach danych. Wartość bliska 1 oznacza dobre zrównoważenie obu metryk.

Krzywa ROC to narzędzie do oceny modeli klasyfikacyjnych, które ilustruje zależność między True Positive Rate (TPR) a False Positive Rate (FPR) dla różnych wartości progu decyzyjnego.



True Positive Rate (Recall):

$$TPR = \frac{TP}{TP + FN}$$

False Positive Rate:

$$FPR = \frac{FP}{FP + TN}$$

AUC to pole pod krzywą ROC i mierzy ogólną skuteczność klasyfikatora:

- $AUC = 1.0$ → Model perfekcyjny.
- $AUC = 0.5$ → Model losowy (brak wartości predykcyjnej).
- $AUC < 0.5$ → Model gorzej niż losowy (należy odwrócić predykcje).

AUC-ROC jest szczególnie przydatne w klasyfikacji, gdy istotne jest równoczesne minimalizowanie FP i FN.

Przykład kodu w Scikit-learn:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, roc_curve, auc, roc_auc_score

accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

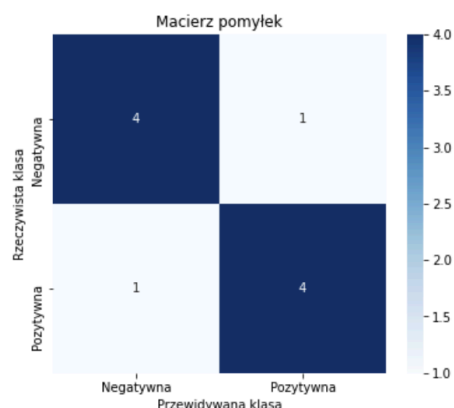
cm = confusion_matrix(y_true, y_pred)

fpr, tpr, thresholds = roc_curve(y_true, y_scores)
auc_score = auc(fpr, tpr)

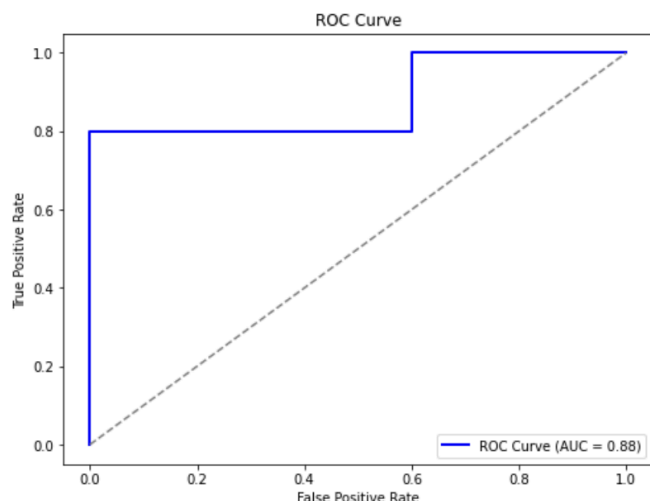
print(f'Accuracy: {accuracy:.4f}')
print(f'Precision: {precision:.4f}')
print(f'Recall: {recall:.4f}')
print(f'F1-score: {f1:.4f}')
print(f'AUC Score: {auc_score:.4f}')
print(f'Confusion Matrix:\n {cm}')
```

```
Accuracy: 0.8000
Precision: 0.8000
Recall: 0.8000
F1-score: 0.8000
AUC Score: 0.8800
Confusion Matrix:
[[4 1]
 [1 4]]
```

```
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Negatywna", "Pozytywna"],
            yticklabels=["Negatywna", "Pozytywna"])
plt.xlabel("Przewidywana klasa")
plt.ylabel("Rzeczywista klasa")
plt.title("Macierz pomyłek")
plt.show()
```



```
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC Curve (AUC = {auc_score:.2f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc='lower right')
plt.show()
```



3. Dobór modeli klasyfikacyjnych

W zależności od charakterystyki danych oraz problemu klasyfikacyjnego należy dobrać odpowiedni model.

Support Vector Machine (SVM)	Skuteczny w klasyfikacji binarnej i wieloklasowej, szczególnie gdy dane mają skomplikowane granice decyzyjne.	Dobrze działa w przestrzeniach wysokiej wymiarowości. Może efektywnie klasyfikować dane nieliniowe dzięki wykorzystaniu funkcji jądra. Odporny na overfitting, jeśli odpowiednio dostrojony.	Wolny dla dużych zbiorów danych. Wymaga dostrojenia parametrów jądra, co może być czasochłonne.
Random Forest	Dobry w klasyfikacji wieloklasowej, problemach z dużą ilością cech, działa dobrze na danych strukturalnych.	Bardzo odporny na overfitting dzięki uśrednianiu wyników wielu drzew decyzyjnych. Działa dobrze nawet na danych z dużą liczbą cech. Wbudowana ocena istotności cech.	Może być wolny dla dużych zbiorów danych. Brak interpretowalności w porównaniu do pojedynczych drzew decyzyjnych.
XGBoost	Bardzo skuteczny dla dużych zbiorów danych.	Wysoka dokładność i wydajność obliczeniowa. Obsługuje dane brakujące. Wbudowane mechanizmy unikania overfittingu (przycinanie drzewa, regularyzacja).	Wrażliwy na dostrojenie hiperparametrów. Może być kosztowny obliczeniowo w przypadku bardzo dużych zbiorów danych.

k-Nearest Neighbors (kNN)	Działa dobrze na małych zbiorach danych, gdzie kluczowa jest lokalna struktura danych.	Intuicyjny i łatwy do zaimplementowania. Dobrze działa w zadaniach, gdzie podobne próbki są blisko siebie w przestrzeni cech.	Bardzo wolny dla dużych zbiorów danych (obliczenia odległości dla każdego punktu w zbiorze). Wrażliwy na wybór metryki odległości.
---	--	---	--

Inne modele możliwe do użycia to [LightGBM](#), [CatBoost](#), [Naive Bayes](#), [AdaBoost](#) itd.

W niektórych przypadkach, zamiast ręcznie wybierać model klasyfikacyjny, można zastosować metodę brute-force, polegającą na przetestowaniu wielu różnych algorytmów i porównaniu ich skuteczności. Takie podejście pozwala na szybkie wybranie najlepszego modelu dla konkretnego zbioru danych bez wcześniejszych założeń na temat jego struktury.

Metoda brute-force:

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.naive_bayes import GaussianNB

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

models = {
    "Random Forest": RandomForestClassifier(),
    "SVM": SVC(probability=True),
    "kNN": KNeighborsClassifier(),
    "XGBoost": XGBClassifier(),
    "Naive Bayes": GaussianNB()
}

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    results[name] = accuracy
    print(f"{name}: Accuracy = {accuracy:.4f}")

best_model = max(results, key=results.get)
print(f"Najlepszy model: {best_model} z dokładnością {results[best_model]:.4f}")
```

Jest to przydatna metoda gdy nie mamy wcześniejszych założeń co do struktury danych i chcemy empirycznie sprawdzić skuteczność różnych podejść.

Najlepiej jednak empirycznie eliminować możliwe na podstawie znanej struktury danych i problemu.

4. Dostrajanie hiperparametrów

Dostrajanie hiperparametrów to etap optymalizacji modeli klasyfikacyjnych, który pozwala na poprawę skuteczności modelu poprzez znalezienie najlepszych wartości parametrów sterujących jego działaniem.

Przykłady modeli z ich hiperparametrami:

Support Vector Machine (SVM)	C – kontroluje kompromis między maksymalizacją marginesu a minimalizacją błędów klasyfikacyjnych. Kernel – wybór funkcji jądra (liniowe, RBF, wielomianowe). Gamma – parametr dla jąder RBF i wielomianowych, wpływa na złożoność granicy decyzyjnej.
Random Forest	n_estimators – liczba drzew w lesie. max_depth – maksymalna głębokość drzew (większe wartości zwiększają ryzyko overfittingu). min_samples_split – minimalna liczba próbek wymagana do podziału węzła. max_features – liczba cech do rozważenia przy podziale węzła (np. sqrt, log2).
XGBoost	learning_rate – tempo uczenia, kontroluje wielkość aktualizacji wag. n_estimators – liczba drzew. max_depth – maksymalna głębokość drzew. subsample – frakcja danych używanych do uczenia każdego drzewa. colsample_bytree – frakcja cech wykorzystywanych w każdym drzewie.
k-Nearest Neighbors (kNN)	n_neighbors (k) – liczba sąsiadów brana pod uwagę przy klasyfikacji. metric – metryka odległości (np. euklidesowa, manhattan, minkowski). weights – sposób ważenia sąsiadów (jednakowe wagi lub odwrotność odległości).

Grid Search (Przeszukiwanie siatki) - polega na przeszukiwaniu wszystkich możliwych kombinacji zadanych wartości hiperparametrów i wybieraniu najlepszego zestawu na podstawie wyników walidacji krzyżowej.

Zalety: Gwarantuje znalezienie optymalnego rozwiązania w określonym zakresie wartości. Prosty do implementacji.

Wady: Może być obliczeniowo kosztowny dla dużych zbiorów danych i szerokiego zakresu hiperparametrów.

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10]
}

model = RandomForestClassifier()
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
print("Najlepsze parametry:", grid_search.best_params_)
```

Random Search (Losowe przeszukiwanie) - losowo wybiera kombinacje wartości hiperparametrów z określonego zakresu, zamiast sprawdzać wszystkie możliwe kombinacje jak w Grid Search.

Zalety: Szybszy niż Grid Search dla dużych przestrzeni hiperparametrów. Może znaleźć dobrą konfigurację bez konieczności przeszukiwania całej siatki.

Wady: Nie daje gwarancji znalezienia optymalnego zestawu parametrów.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_dist = {
    'n_estimators': randint(50, 200),
    'max_depth': [None, 10, 20],
    'min_samples_split': randint(2, 10)
}

random_search = RandomizedSearchCV(model, param_dist, n_iter=10, cv=5, scoring='accuracy', random_state=42)
random_search.fit(X_train, y_train)
print("Najlepsze parametry:", random_search.best_params_)
```

Bayesian Optimization (Optymalizacja Bayesowska) - W przeciwieństwie do Grid Search i Random Search, optymalizacja Bayesowska wykorzystuje model probabilistyczny do iteracyjnego wybierania najlepszych wartości hiperparametrów, zamiast testowania ich losowo lub sekwencyjnie.

Zalety: Wymaga mniej iteracji niż Grid Search, aby znaleźć dobre wartości hiperparametrów. Lepsza efektywność obliczeniowa dla dużych zbiorów danych.

Wady: Bardziej skomplikowana implementacja niż Grid Search czy Random Search.

Popularna biblioteka do optymalizacji Bayesowskiej: optuna.

```
import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

def objective(trial):
    n_estimators = trial.suggest_int('n_estimators', 50, 200)
    max_depth = trial.suggest_categorical('max_depth', [None, 10, 20])
    min_samples_split = trial.suggest_int('min_samples_split', 2, 10)

    model = RandomForestClassifier(n_estimators=n_estimators,
                                  max_depth=max_depth,
                                  min_samples_split=min_samples_split)
    score = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy').mean()
    return score

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=20)
print("Najlepsze parametry:", study.best_params_)
```


5. Analiza wyników modeli i analiza istotności cech

Po dostrojeniu hiperparametrów i wyborze modelu należy przeanalizować jego skuteczność. Analiza wyników obejmuje ocenę modeli na podstawie różnych metryk oraz interpretację ich wpływu na klasyfikację.

Analiza wyników:

- Porównanie różnych modeli – wykorzystanie metryk (accuracy, precision, recall, F1-score, AUC-ROC) do wyboru najlepszego modelu.
- Ocena błędów klasyfikacji – analiza wartości w macierzy pomyłek.
- Analiza wpływu niezbalansowanych danych – sprawdzenie, czy model nie faworyzuje jednej klasy kosztem drugiej.
- Interpretacja wartości predykcyjnych – weryfikacja poprawności przewidywań w kontekście dziedziny problemu.

Analiza istotności cech pozwala określić, które zmienne najbardziej wpływają na predykcję modelu. Dzięki temu możemy:

- **Eliminować nieistotne cechy**, co może poprawić generalizację modelu i zmniejszyć ryzyko overfittingu.
- **Zrozumieć strukturę danych** – określić, które zmienne mają największy wpływ na wyniki klasyfikacji.
- **Optymalizować model** – uprościć model i poprawić jego interpretowalność.

W modelach opartych na drzewach, takich jak **Random Forest** i **XGBoost**, znaczenie cech można oszacować na podstawie liczby razy, gdy dana cecha została użyta do podziału węzła:

```
from sklearn.ensemble import RandomForestClassifier
import matplotlib.pyplot as plt
import numpy as np

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

importances = model.feature_importances_
feature_names = X_train.columns

indices = np.argsort(importances)[::-1]
plt.figure(figsize=(8, 5))
plt.title("Feature Importance")
plt.bar(range(X_train.shape[1]), importances[indices], align="center")
plt.xticks(range(X_train.shape[1]), feature_names[indices], rotation=90)
plt.xlabel("Cechy")
plt.ylabel("Istotność")
plt.show()
```


[SHAP](#) to bardziej zaawansowana metoda interpretacji modeli ML, pozwalająca na analizę wpływu każdej cechy na poszczególne predykcje:

```
import shap

explainer = shap.Explainer(model, X_train)
shap_values = explainer(X_test)

shap.summary_plot(shap_values, X_test)
```

Dla modeli liniowych, takich jak SVM (z liniowym jądrem) czy regresja Lasso, istotność cech można ocenić na podstawie wag współczynników:

```
from sklearn.svm import SVC

model = SVC(kernel='linear')
model.fit(X_train, y_train)

print("Współczynniki cech:", model.coef_)
```

6. Zadanie

Wykorzystując zbiór danych oraz model bazowy stworzony podczas pierwszego laboratorium, przeprowadź proces optymalizacji klasyfikacji. Skoncentruj się na doborze najlepszego modelu, dostrojeniu jego hiperparametrów, ocenie jakości oraz analizie wyników.

W pierwszym kroku przeprowadź testowanie różnych modeli klasyfikacyjnych na przygotowanym zbiorze danych, porównując ich skuteczność. Oceń skuteczność modeli na podstawie metryk. Po wyborze najlepszego modelu przeprowadź dostrajanie jego hiperparametrów.

Po optymalizacji przeprowadź szczegółową analizę jakości modelu. Na zakończenie wykonaj analizę istotności cech w modelu, jeśli to możliwe.

Sporządź wnioski i obserwacje na temat działania modelu.